Decoder-free reproductions of magic-state simulations with real non-Clifford gates on a GPU

Generated by `repro/make_report.py`

2026-11-05 07:37

Every figure below is regenerated from scratch by `uv run python -m repro.run_all`: circuits come from the authors' generators, are sampled with the Tsim CUDA statevector executor (postselection on the device), and every point records its own GPU time, compile time, shot count, GPU model and Tsim build. Where the paper simulated a Clifford proxy (S in place of T, or a $|Y\rangle$ / $|+i\rangle$ state), the same proxy is sampled again on the GPU and plotted next to the published numbers, as a check of the GPU kernel; the real non-Clifford circuit (T gates, rotations) is sampled alongside. Logical error rates are per kept shot; bands and error bars span hypotheses within a factor 1000 of the maximum likelihood (the convention of `sinter`). Triangles mark upper bounds for points with zero observed errors.

Tsim builds: a2af975fbc.

study	points	GPU sampling	shots
gidney_cultivation	32	14.7 min	2.4e12
wan_few_clifford_terms	16	4.5 min	6.5e11
soft_d5	1	12 s	3.3e10
clifft_inject_cultivate	0	0.0 s	0
gauging_cultivation	1	1.0 min	1.5e11
fold_transversal	0	0.0 s	0
chan_flags	0	0.0 s	0
magna	0	0.0 s	0
google_experiment	0	0.0 s	0
star_injection	0	0.0 s	0
rp2_ungrown	0	0.0 s	0
erasure_injection	0	0.0 s	0
vaknin_full_vector	0	0.0 s	0

Contents

Reproduction checks **3**

1 Gidney, Shutty, Jones (2024), Magic state cultivation: growing T states as cheap as CNOT gates **5**

2 Wan, Zhong, Zapirain (2025), Simulating magic state cultivation with few Clifford terms **7**

3 Li et al. (2025), SOFT: a high-performance simulator for universal fault-tolerant quantum circuits **8**

4 Chase, Labib (2026), Clifft: fast exact simulation of near-Clifford quantum circuits **9**

5 Hetényi, Brown, Williamson (2026), Constant-depth magic state cultivation with Clifford measurements by gauging **10**

6 Sahay et al. (2025), Fold-transversal surface code cultivation **11**

7 Chan, Strikis, Sun, Cai (2026), Diagnosing and restoring the degraded fault distance of magic state cultivation 12

8 Wang, Gu (2026), Magna: simulating magic state factories as cheaply as Clifford circuits 14

9 Rosenfeld et al. (2025), Magic state cultivation on a superconducting quantum processor 16

10 Ismail et al. (2025), Transversal architecture for megaquop-scale quantum simulation with neutral atoms (transversal STAR) 18

11 Chen et al. (2025), Efficient magic state cultivation on RP^2 19

12 Jacoby et al. (2025), Magic state injection with erasure qubits 20

13 Vaknin et al. (2025), Efficient magic state cultivation on the surface code 21

Reproduction checks against published numbers

Every published number (tables, released data) that a sampled point can be compared with. ±: one standard deviation (Poisson for error counts, binomial for acceptance; published values without an uncertainty, or exact values, have none). z: difference over the combined standard deviation.

study	point	variant	quantity	published	Tsim GPU	ratio	z
gidney_cultivation	bell d=3 p=0.0005 S (Stim)	clifford	LER	4.91e-08 ± 2e-09	4.89e-08 ± 3e-10	0.996	-0.1
gidney_cultivation	bell d=3 p=0.0005 S (Stim)	clifford	acceptance	0.846 ± 2e-06	0.846 ± 5e-07	1.000	+1.2
gidney_cultivation	bell d=3 p=0.001 S (Stim)	clifford	LER	3.92e-07 ± 1e-08	3.99e-07 ± 3e-09	1.019	+0.6
gidney_cultivation	bell d=3 p=0.001 S (Stim)	clifford	acceptance	0.716 ± 7e-06	0.716 ± 2e-06	1.000	+2.1
gidney_cultivation	bell d=3 p=0.002 S (Stim)	clifford	LER	3.25e-06 ± 1e-07	3.32e-06 ± 2e-08	1.020	+0.6
gidney_cultivation	bell d=3 p=0.002 S (Stim)	clifford	acceptance	0.513 ± 2e-05	0.513 ± 5e-06	1.000	-1.5
gidney_cultivation	teleport d=3 p=0.0005 S (Stim)	clifford	LER	7.28e-08 ± 2e-09	7.12e-08 ± 5e-10	0.978	-0.7
gidney_cultivation	teleport d=3 p=0.0005 S (Stim)	clifford	acceptance	0.804 ± 3e-06	0.804 ± 7e-07	1.000	-1.7
gidney_cultivation	teleport d=3 p=0.001 S (Stim)	clifford	LER	5.79e-07 ± 2e-08	5.89e-07 ± 4e-09	1.018	+0.6
gidney_cultivation	teleport d=3 p=0.001 S (Stim)	clifford	acceptance	0.646 ± 9e-06	0.646 ± 2e-06	1.000	-0.9
gidney_cultivation	teleport d=3 p=0.002 S (Stim)	clifford	LER	4.81e-06 ± 2e-07	4.83e-06 ± 3e-08	1.005	+0.2
gidney_cultivation	teleport d=3 p=0.002 S (Stim)	clifford	acceptance	0.418 ± 2e-05	0.418 ± 5e-06	1.000	-2.3
gidney_cultivation	unitary d=3 p=0.0005 S (Stim)	clifford	LER	4e-08 ± 1e-09	4.11e-08 ± 3e-10	1.027	+0.9
gidney_cultivation	unitary d=3 p=0.0005 S (Stim)	clifford	acceptance	0.829 ± 2e-06	0.829 ± 5e-07	1.000	-1.6
gidney_cultivation	unitary d=3 p=0.001 S (Stim)	clifford	LER	3.42e-07 ± 1e-08	3.43e-07 ± 2e-09	1.001	+0.0
gidney_cultivation	unitary d=3 p=0.001 S (Stim)	clifford	acceptance	0.687 ± 7e-06	0.687 ± 2e-06	1.000	+1.0
gidney_cultivation	unitary d=3 p=0.001 T (state vector)	t	LER	9.73e-07 ± 5e-08	8.98e-07 ± 6e-09	0.923	-1.4
gidney_cultivation	unitary d=3 p=0.001 T (state vector)	t	acceptance	0.687 ± 2e-05	0.687 ± 3e-06	1.000	+0.7
gidney_cultivation	unitary d=3 p=0.001 S (state vector)	clifford	LER	3.89e-07 ± 3e-08	3.43e-07 ± 2e-09	0.880	-1.4
gidney_cultivation	unitary d=3 p=0.001 S (state vector)	clifford	acceptance	0.687 ± 2e-05	0.687 ± 2e-06	1.000	-1.4
gidney_cultivation	unitary d=3 p=0.001 Z (state vector)	clifford	LER	2.93e-07 ± 3e-08	2.88e-07 ± 2e-09	0.983	-0.2
gidney_cultivation	unitary d=3 p=0.001 Z (state vector)	clifford	acceptance	0.687 ± 2e-05	0.687 ± 1e-06	1.000	+2.2
gidney_cultivation	unitary d=3 p=0.002 S (Stim)	clifford	LER	2.85e-06 ± 9e-08	2.95e-06 ± 2e-08	1.037	+1.1
gidney_cultivation	unitary d=3 p=0.002 S (Stim)	clifford	acceptance	0.472 ± 2e-05	0.472 ± 4e-06	1.000	+0.3
gidney_cultivation	unitary d=3 p=0.002 T (state vector)	t	LER	5.86e-06 ± 1e-07	5.69e-06 ± 4e-08	0.972	-1.1
gidney_cultivation	unitary d=3 p=0.002 T (state vector)	t	acceptance	0.472 ± 2e-05	0.472 ± 6e-06	1.000	-0.3
gidney_cultivation	unitary d=3 p=0.002 S (state vector)	clifford	LER	2.87e-06 ± 1e-07	2.95e-06 ± 2e-08	1.028	+0.7
gidney_cultivation	unitary d=3 p=0.002 S (state vector)	clifford	acceptance	0.472 ± 2e-05	0.472 ± 4e-06	1.000	-1.1
gidney_cultivation	unitary d=3 p=0.002 Z (state vector)	clifford	LER	2.53e-06 ± 1e-07	2.46e-06 ± 2e-08	0.973	-0.7
gidney_cultivation	unitary d=3 p=0.002 Z (state vector)	clifford	acceptance	0.472 ± 2e-05	0.472 ± 4e-06	1.000	-1.1
gidney_cultivation	unitary d=3 p=0.003 T (state vector)	t	LER	1.81e-05 ± 6e-07	1.81e-05 ± 1e-07	1.000	+0.0
gidney_cultivation	unitary d=3 p=0.003 T (state vector)	t	acceptance	0.325 ± 4e-05	0.325 ± 8e-06	1.000	+1.2
gidney_cultivation	unitary d=3 p=0.003 S (state vector)	clifford	LER	1.03e-05 ± 4e-07	1.05e-05 ± 7e-08	1.027	+0.7
gidney_cultivation	unitary d=3 p=0.003 S (state vector)	clifford	acceptance	0.325 ± 3e-05	0.325 ± 6e-06	1.000	-1.4
gidney_cultivation	unitary d=3 p=0.003 Z (state vector)	clifford	LER	9.28e-06 ± 4e-07	9e-06 ± 6e-08	0.970	-0.7
gidney_cultivation	unitary d=3 p=0.003 Z (state vector)	clifford	acceptance	0.325 ± 3e-05	0.325 ± 6e-06	1.000	+1.1
gidney_cultivation	unitary d=3 p=0.005 T (state vector)	t	LER	8.75e-05 ± 3e-06	8.48e-05 ± 4e-07	0.969	-1.0
gidney_cultivation	unitary d=3 p=0.005 T (state vector)	t	acceptance	0.155 ± 4e-05	0.155 ± 6e-06	0.999	-2.6
gidney_cultivation	unitary d=3 p=0.005 S (state vector)	clifford	LER	5.81e-05 ± 2e-06	5.69e-05 ± 3e-07	0.980	-0.6
gidney_cultivation	unitary d=3 p=0.005 S (state vector)	clifford	acceptance	0.155 ± 3e-05	0.155 ± 6e-06	1.000	+0.6
gidney_cultivation	unitary d=3 p=0.005 Z (state vector)	clifford	LER	4.92e-05 ± 2e-06	4.84e-05 ± 3e-07	0.983	-0.5
gidney_cultivation	unitary d=3 p=0.005 Z (state vector)	clifford	acceptance	0.155 ± 3e-05	0.155 ± 6e-06	1.000	-0.3
gidney_cultivation	unitary d=3 p=0.007 T (state vector)	t	LER	0.00026 ± 8e-06	0.000252 ± 2e-06	0.970	-0.9
gidney_cultivation	unitary d=3 p=0.007 T (state vector)	t	acceptance	0.0741 ± 4e-05	0.0741 ± 7e-06	1.001	+1.3
gidney_cultivation	unitary d=3 p=0.007 S (state vector)	clifford	LER	0.00019 ± 6e-06	0.000184 ± 7e-07	0.966	-1.1
gidney_cultivation	unitary d=3 p=0.007 S (state vector)	clifford	acceptance	0.0741 ± 3e-05	0.0742 ± 3e-06	1.000	+0.6
gidney_cultivation	unitary d=3 p=0.007 Z (state vector)	clifford	LER	0.000154 ± 5e-06	0.000154 ± 6e-07	1.003	+0.1
gidney_cultivation	unitary d=3 p=0.007 Z (state vector)	clifford	acceptance	0.0743 ± 3e-05	0.0743 ± 3e-06	1.000	+0.9
gidney_cultivation	unitary d=3 p=0.01 T (state vector)	t	LER	0.00091 ± 3e-05	0.000866 ± 5e-06	0.951	-1.5
gidney_cultivation	unitary d=3 p=0.01 T (state vector)	t	acceptance	0.0249 ± 2e-05	0.0249 ± 4e-06	0.998	-1.7
gidney_cultivation	unitary d=3 p=0.01 S (state vector)	clifford	LER	0.000667 ± 2e-05	0.000685 ± 4e-06	1.028	+0.9
gidney_cultivation	unitary d=3 p=0.01 S (state vector)	clifford	acceptance	0.0249 ± 2e-05	0.0249 ± 4e-06	1.001	+1.4
gidney_cultivation	unitary d=3 p=0.01 Z (state vector)	clifford	LER	0.000573 ± 2e-05	0.000585 ± 4e-06	1.021	+0.7
gidney_cultivation	unitary d=3 p=0.01 Z (state vector)	clifford	acceptance	0.025 ± 2e-05	0.025 ± 4e-06	1.000	+0.4
wan_few_clifford_terms	d=3 p=0.0005 T	t	LER	1.8e-07 ± 7e-09	1.83e-07 ± 1e-09	1.022	+0.5
wan_few_clifford_terms	d=3 p=0.0005 T	t	acceptance	0.807 ± 6e-06	0.804 ± 1e-06	0.997	-375.0
wan_few_clifford_terms	d=3 p=0.001 T	t	LER	1e-06 ± 5e-08	1.06e-06 ± 7e-09	1.055	+1.0
wan_few_clifford_terms	d=3 p=0.001 T	t	acceptance	0.651 ± 2e-05	0.647 ± 3e-06	0.994	-179.4
wan_few_clifford_terms	d=3 p=0.0015 T	t	LER	2.95e-06 ± 1e-07	3.09e-06 ± 2e-08	1.049	+1.4
wan_few_clifford_terms	d=3 p=0.0015 T	t	acceptance	0.525 ± 2e-05	0.521 ± 4e-06	0.991	-204.1
wan_few_clifford_terms	d=3 p=0.002 T	t	LER	6.76e-06 ± 2e-07	6.94e-06 ± 5e-08	1.027	+1.0
wan_few_clifford_terms	d=3 p=0.002 T	t	acceptance	0.424 ± 2e-05	0.419 ± 6e-06	0.989	-217.6
wan_few_clifford_terms	d=3 p=0.0025 T	t	LER	1.29e-05 ± 3e-07	1.28e-05 ± 9e-08	0.992	-0.4

study	point	variant	quantity	published	Tsim GPU	ratio	z
wan_few_clifford_terms	d=3 p=0.0025 T	t	acceptance	0.343 ± 2e-05	0.338 ± 7e-06	0.986	-224.8
wan_few_clifford_terms	d=3 p=0.003 T	t	LER	2.15e-05 ± 4e-07	2.17e-05 ± 2e-07	1.008	+0.4
wan_few_clifford_terms	d=3 p=0.003 T	t	acceptance	0.277 ± 2e-05	0.272 ± 8e-06	0.983	-227.5
wan_few_clifford_terms	d=3 p=0.004 T	t	LER	4.93e-05 ± 7e-07	5.08e-05 ± 3e-07	1.030	+1.9
wan_few_clifford_terms	d=3 p=0.004 T	t	acceptance	0.181 ± 2e-05	0.177 ± 7e-06	0.978	-222.8
wan_few_clifford_terms	d=3 p=0.005 T	t	LER	9.77e-05 ± 1e-06	9.9e-05 ± 5e-07	1.012	+0.9
wan_few_clifford_terms	d=3 p=0.005 T	t	acceptance	0.118 ± 1e-05	0.115 ± 6e-06	0.972	-219.7
gauging_cultivation	d=3 p=0.0005 |+i> proxy	clifford	LER	1.2e-07 ± 2e-08	1.09e-07 ± 1e-09	0.911	-0.5
gauging_cultivation	d=3 p=0.0005 |+i> proxy	clifford	acceptance	0.65	0.679 ± 1e-06	1.044	+23372.6

1 Gidney, Shutty, Jones (2024), Magic state cultivation: growing T states as cheap as CNOT gates

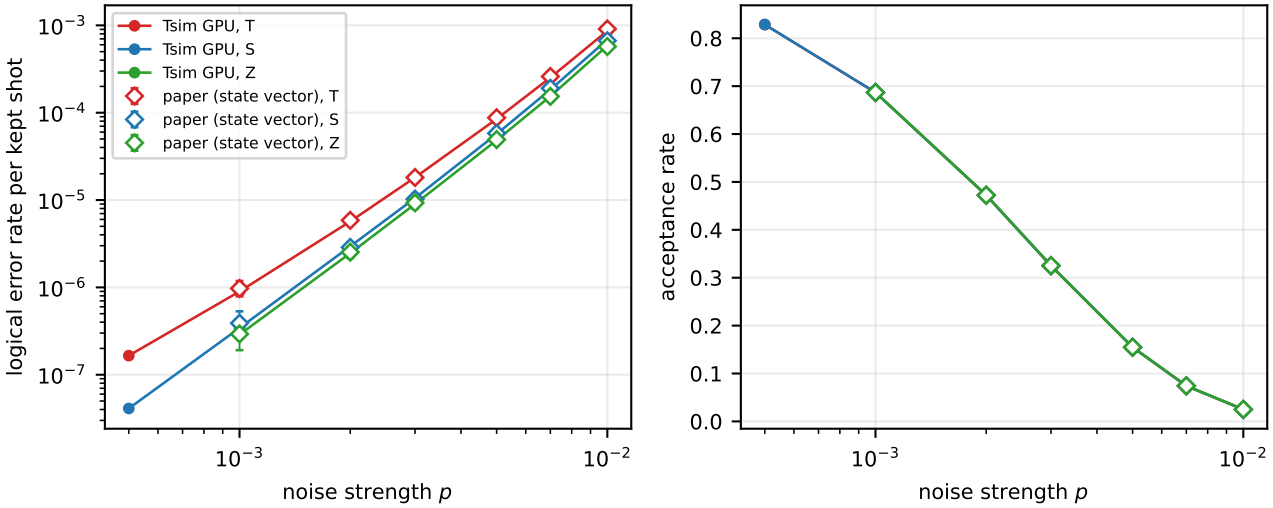
arXiv:2409.17595 Study module `repro/studies/gidney_cultivation.py`

Two figures, both decoder-free (all detectors postselected, noiseless final logical check):

- Fig. 13 (`t-vs-s`): unitary injection + d=3 cultivation with each T gate replaced by T, S or Z, vs. uniform depolarizing noise p. The paper used a state-vector simulator ("vec_intercept").
- Fig. 21 (`inject-only-sampled`): bell / teleport ("degenerate") / unitary injection + d=3 cultivation, and unitary + d=5 cultivation, at p = 5e-4, 1e-3, 2e-3. The paper sampled the S proxy with Stim ("perfectionist" decoder = discard on any detection event).

Circuits: the authors' generator (`cultiv.make_inject_and_cultivate_circuit`, commit 871e68f) with their `gen.NoiseModel.uniform_depolarizing(p)`; T circuits by the Clifft paper's converter (`convert_circuit`: S->T, d=5 sign flips and feedforward, final Y_L wrapped in T...T_DAG). Z variant: S/S_DAG -> Z and the final logical check measures X_L instead of Y_L.

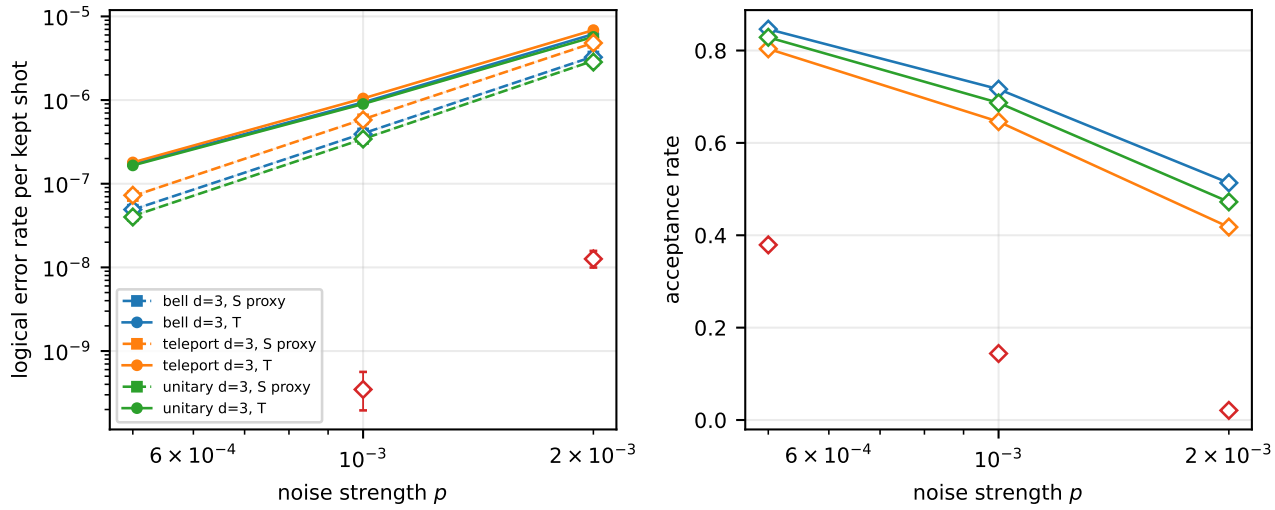
Published numbers overlaid: Zenodo 13777072 `stats.csv`.



Gidney et al. Fig. 13: cultivating T, S, Z states ($d = 3$, no escape)

Compute: 1.6 min GPU sampling + 5.3 s compile, 3.1e11 shots, 18 points (NVIDIA GeForce RTX 5090, Tsim a2af975f)

Unitary injection followed by $d = 3$ cultivation; every detector postselected; noiseless final logical check (for T: $T^\dagger$-rotated Y_L). Open markers: the paper's state-vector simulation (Zenodo stats.csv). Filled markers: Tsim GPU. S and Z are Clifford proxies of the same circuit.



Gidney et al. Fig. 21: injection + cultivation, sampled
Compute: 14.0 min GPU sampling + 6.0 s compile, 2.2e12 shots, 18 points (NVIDIA GeForce RTX 5090, Tsim a2af975f)
Three injection styles with $d = 3$ cultivation and unitary injection with $d = 5$ cultivation, all detectors postselected. Dashed/squares: the S proxy (the paper's Clifford simulation); solid/circles: real T gates. Open diamonds: the paper's Stim data (Zenodo stats.csv, up to 10^{12} shots per point). The right panel shows discard rates of the S proxy.

Gidney et al.: $d = 5$ injection + cultivation (no escape)
Each cell: logical error rate of kept shots, errors / kept shots, discard rate, and (Tsim) GPU sampling time.
Compute: not yet sampled

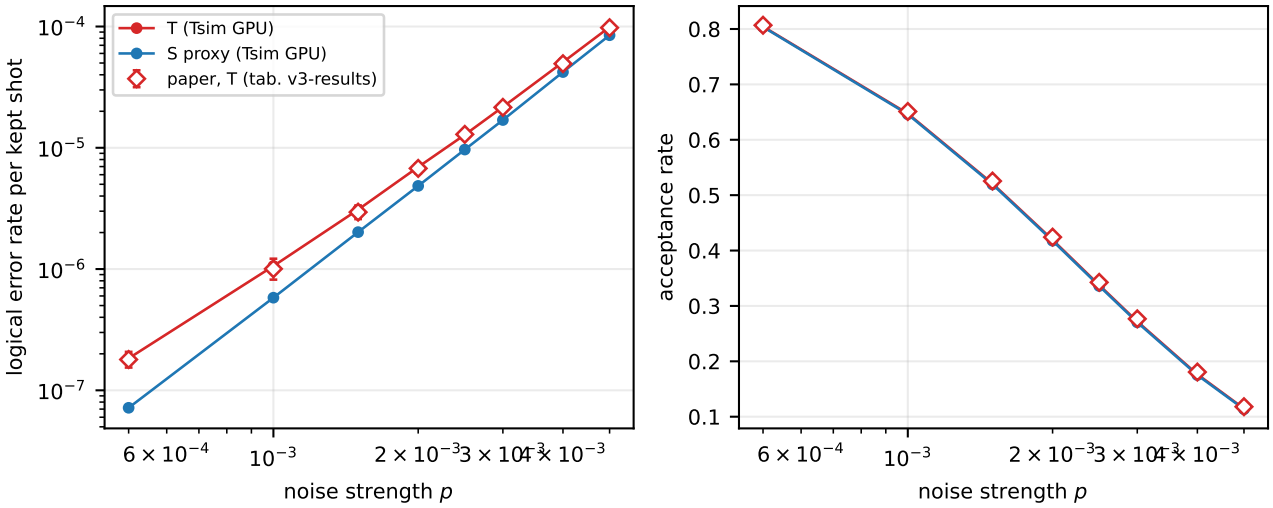
p	paper (S proxy, Stim)	Tsim GPU, S proxy	Tsim GPU, T
0.0005	0 (0/3.8e11, disc. 0.6208)	–	–
0.001	3.47e-10 (50/1.4e11, disc. 0.8560)	–	–
0.002	1.26e-08 (263/2.1e10, disc. 0.9792)	–	–

2 Wan, Zhong, Zapirain (2025), Simulating magic state cultivation with few Clifford terms

arXiv:2509.08658 Study module `repro/studies/wan_few_clifford_terms.py`

The paper samples the native-T $d = 3$ cultivation circuit (injection + double check, from the companion repo's `d_3_circuit_definitions.py`) followed by a noiseless T-projection readout, postselecting on every detector. Its Table (`tab:v3-results`) lists the exact non-Clifford results at 2^{29}–2^{32} shots per noise strength; its red curves are the S-gate proxy sampled with Stim.

Circuits: built as in the repo's `run.py` and our `make_circuits.py`: $T \to S$, the vendored `gen.NoiseModel.uniform_depolar` $S \to T$; the projection stays noiseless. The Clifford proxy maps every $T/T^\dagger$ of the whole circuit (including the projection) to $S/S^\dagger$.



Wan et al. Figs. 15–17, 19: $d = 3$ T cultivation, LER and post-selection rate
Compute: 4.5 min GPU sampling + 5.3 s compile, 6.5e11 shots, 16 points (NVIDIA GeForce RTX 5090, Tsim a2af975f)
Injection + double check with a noiseless T-projection readout; every detector postselected. Open diamonds: the paper's exact non-Clifford results (Table v3-results, 2^{29}–2^{32} shots, Apple M4 Pro CPU). Red: real T gates; blue: the S proxy (the paper's Stim reference).

Wan et al. Table v3-results vs. Tsim GPU

Compute: 1.3 min GPU sampling + 3.0 s compile, 2e11 shots, 8 points (NVIDIA GeForce RTX 5090, Tsim a2af975f)

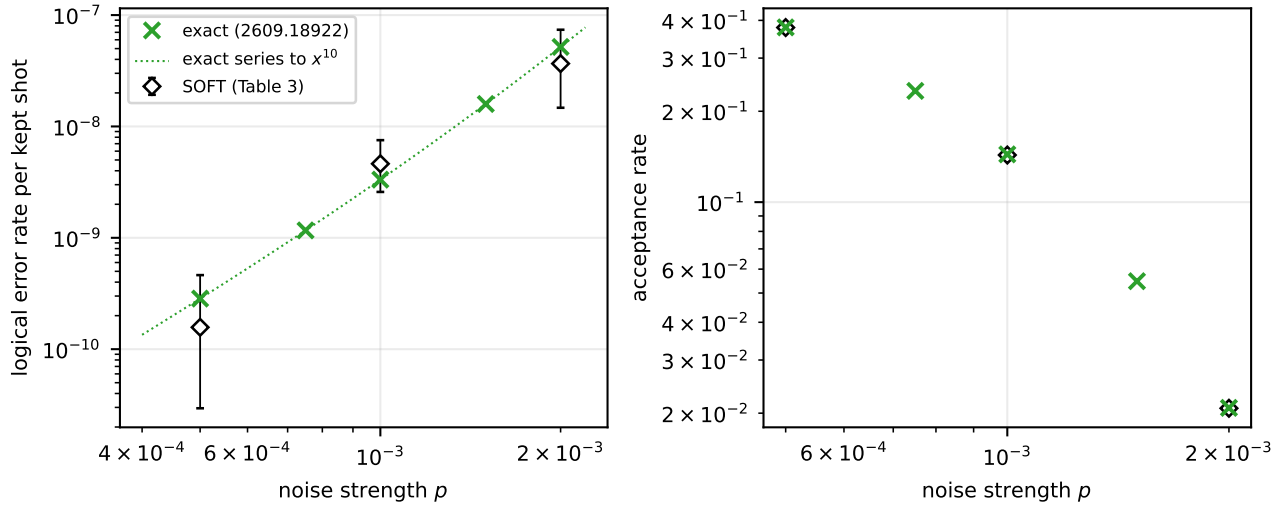
	paper (T, CPU)				Tsim GPU (T)				
p	LER	PSR	errors	shots	LER	PSR	errors	shots	time
0.0005	1.8e-07	0.807	622	4.3e9	1.83e-07	0.8045	20029	1.4e11	60 s
0.001	1e-06	0.651	351	5.4e8	1.06e-06	0.6473	20034	2.9e10	11 s
0.0015	2.9e-06	0.525	831	5.4e8	3.09e-06	0.5210	20479	1.3e10	4.1 s
0.002	6.8e-06	0.424	1540	5.4e8	6.94e-06	0.4194	20425	7e9	2.1 s
0.0025	1.3e-05	0.343	2373	5.4e8	1.28e-05	0.3377	20794	4.8e9	1.3 s
0.003	2.2e-05	0.277	3201	5.4e8	2.17e-05	0.2720	20768	3.5e9	1.0 s
0.004	4.9e-05	0.181	4785	5.4e8	5.08e-05	0.1766	23758	2.6e9	0.8 s
0.005	9.8e-05	0.118	6193	5.4e8	9.9e-05	0.1147	37503	3.3e9	0.8 s

3 Li et al. (2025), SOFT: a high-performance simulator for universal fault-tolerant quantum circuits

arXiv:2512.23037 Study module `repro/studies/soft_d5.py`

SOFT sampled the authors' own native-T circuits (unitary injection, $d = 3$ and $d = 5$ double checks, code growth, noiseless readout; every detector postselected) on 16 H800 GPUs for 20 days: 28.9B, 74B and 134.4B shots at p = 2e-3, 1e-3, 5e-4. SOFT ran no Clifford proxy, so only the non-Clifford version is sampled here (the S proxy of the same protocol is in the Gidney study).

Circuits: SOFT's released files (`softv1_magic_state_cultivation`), used verbatim. They are also the reference circuits of the exact-LER paper (Wan, Zapirain, arXiv:2609.18922), whose exact values $A(p)$ and $P_L(p)$ and truncated series are overlaid.



SOFT Fig. 2 / Table 3: $d = 5$ injection + cultivation with real T gates
Compute: not yet sampled
SOFT's own circuits (verbatim), all detectors postselected, noiseless final check. Black diamonds: SOFT's published values (16 H800 GPUs, 20 days, 2.4×10^{11} shots). Green crosses and dotted line: exact values and series of Wan and Zapirain (arXiv:2609.18922) for the same circuits.

SOFT Table 3 vs. exact values vs. Tsim GPU
LER of kept shots (errors / kept); A is the acceptance rate; Tsim brackets are 1000x likelihood intervals.
Compute: 12 s GPU sampling + 0.3 s compile, 3.3e10 shots, 1 points (NVIDIA GeForce RTX 5090, Tsim a2af975f)

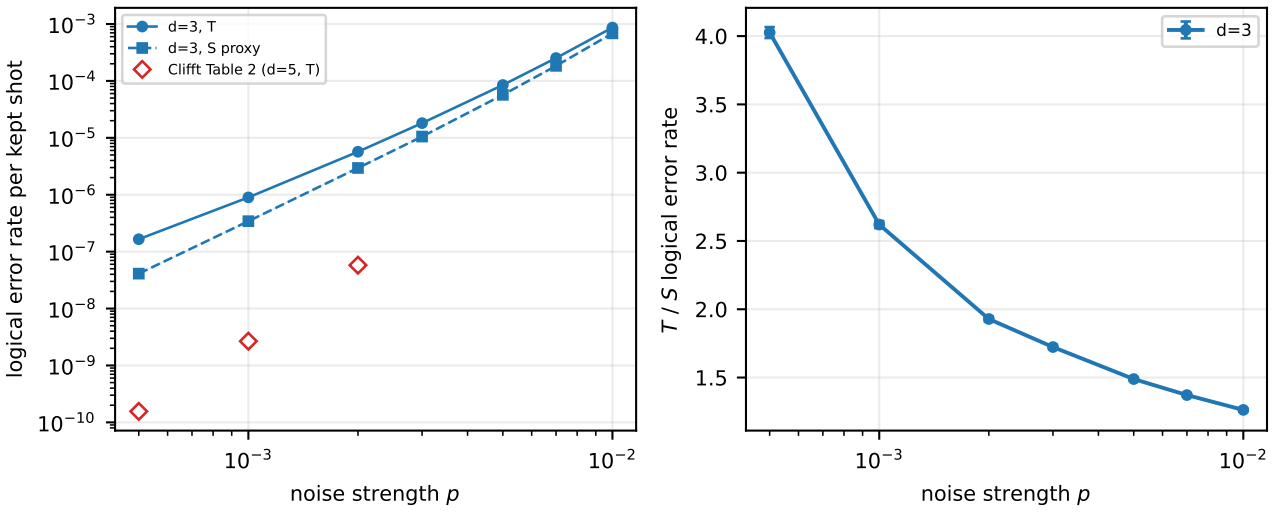
p	SOFT (Table 3)	exact (2609.18922)	Tsim GPU	shots, GPU time
0.0005	1.57e-10 (8/5.1e10)	2.846e-10, A=0.3792	–	–
0.001	4.62e-09 (49/1.1e10)	3.328e-09, A=0.1440	–	–
0.002	3.67e-08 (22/6e8)	5.175e-08, A=0.0208	–	–
0.003	–	–	–	–
0.004	–	–	–	–
0.005	–	–	–	–

4 Chase, Labib (2026), Clifft: fast exact simulation of near-Clifford quantum circuits

arXiv:2604.27058 Study module `repro/studies/clifft_inject_cultivate.py`

Clifft's Fig. 6 plots the logical error rate per kept shot of Gidney's unitary-injection cultivation circuits (no escape) with real T gates and with the S proxy, and their ratio; its Table 2 lists the $d = 5$ T-gate values at three noise strengths (stratified importance sampling, 9.6×10^{10} shots, 12 machine-hours on a 16-core c6i.8xlarge).

Clifft's circuits are the same as the Gidney study's (the Clifft paper's converter, byte-identical at $p = 10^{-3}$), so this study samples nothing itself: it re-plots the Gidney study's unitary-injection points as LER and T/S ratio.



Clifft Fig. 6: inject + cultivate, T vs. S proxy, and their ratio

Compute: (Gidney study points): 5.6 min GPU sampling + 4.0 s compile, 9e11 shots, 14 points (NVIDIA GeForce RTX 5090, Tsim a2af975f)

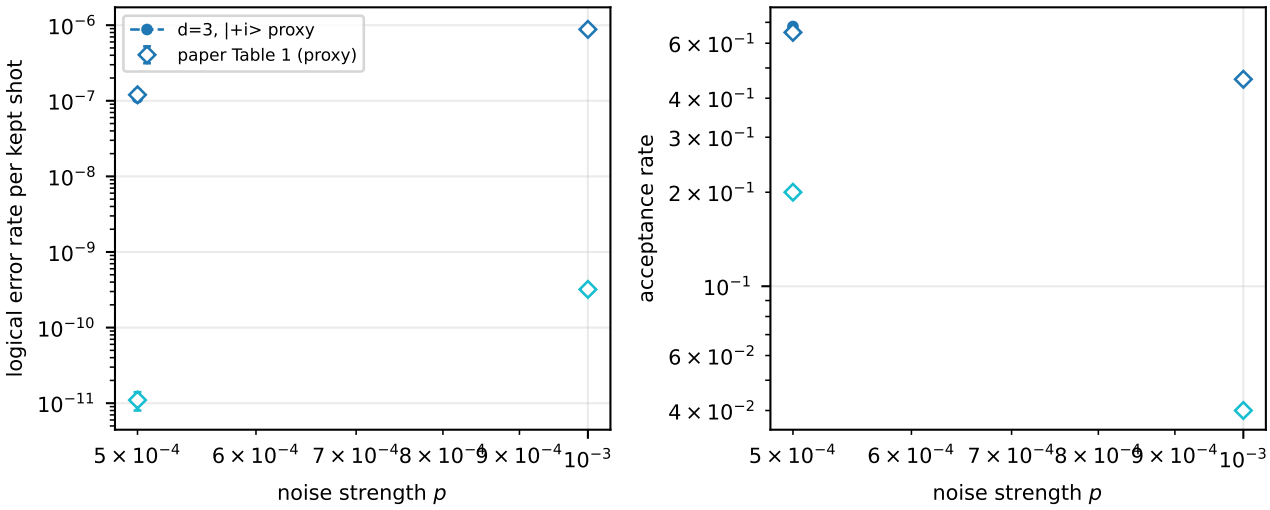
Same samples as the Gidney study (unitary injection). Error bars on the ratio are $\pm 1\sigma$ (Poisson). Open diamonds: Clifft's Table 2 $d = 5$ values (stratified importance sampling).

5 Hetényi, Brown, Williamson (2026), Constant-depth magic state cultivation with Clifford measurements by gauging

arXiv:2603.05429 Study module `repro/studies/gauging_cultivation.py`

Fig. 4(a) compares, at $d = 3$ and $p = 0.1$–0.5%, the authors' Stim simulation of the $|+i\rangle$ proxy with a qiskit state-vector simulation of the T circuit; Table 1 lists P_L and p_{succ} for fault distances 3 and 5 at $p =$ 1e-3 and 5e-4 (the $d = 7$ entries are skipped here, as agreed). Every detector is postselected, including a final noiseless stabilizer round.

Circuits: the authors' generator `get_stim_circ_w_growing(d, basis="Y", pipeline=True)` with their `utils.add_noise` (uniform p), as in their notebook. Clifford proxy: as generated, final noiseless Y_L readout. T version: every $S/S^\dagger$ becomes $T/T^\dagger$ (as in the authors' qiskit check), and a noiseless logical rotation $\exp(\mp i\pi Z_L/8)$ (`repro/magic_check.py`) is inserted before the readout so that Y_L checks the T state deterministically, the same observable as the paper's per-shot $XS^\dagger$ expectation.



Hetényi et al. Fig. 4(a,b) and Table 1: gauging cultivation, $d = 3$ **and** $d = 5$

Compute: 1.0 min GPU sampling + 0.7 s compile, 1.5e11 shots, 1 points (NVIDIA GeForce RTX 5090, Tsim a2af975f)

Full postselection including the final noiseless stabilizer round (counted in the success probability). Dashed: the $|+i\rangle$ Clifford proxy; solid: real T gates with a noiseless T-basis readout. Open diamonds: the paper's Table 1 (proxy, Stim). The $d = 7$ point of the paper is not attempted.

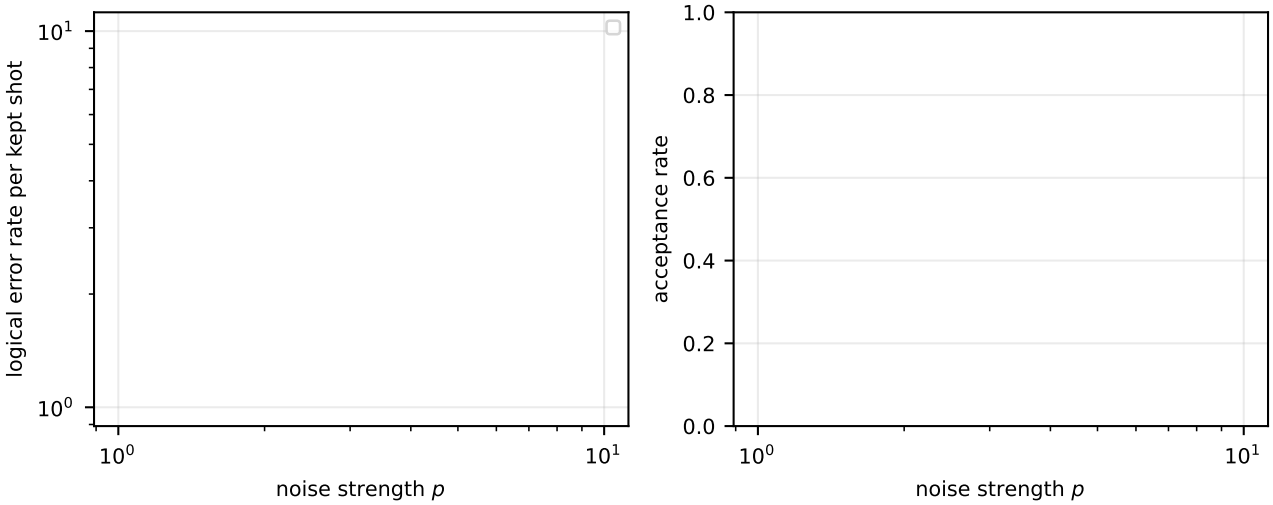
6 Sahay et al. (2025), Fold-transversal surface code cultivation

arXiv:2509.05212 Study module `repro/studies/fold_transversal.py`

The paper compares $|Y\rangle$-state cultivation (Clifford, Stim) with the true $|H_{XY}\rangle = T|+\rangle$ cultivation (a state-vector simulation) on the fault-distance-3 surface-code protocol: unitary preparation plus stabilizer round, two fold-transversal H_{XY} checks with GHZ(3) ancillas, then a noiseless decoding of the code and a logical readout; every detector postselected.

Circuits:

- Clifford proxy: the authors' `generate_sv_cirquit(p, dfinal=7, prep="unitstab", cirq=False)` (kaavyas99/MSC_ at 9378fd2), the circuit their state-vector handoff starts from; noiseless logical Y readout.
- T: the native-T reconstruction of the same circuit by Wan and Zapirain (arXiv:2609.18922, `fold-d3/model.json`: CY layers -> CCZ + controlled-SX), exported by our `make_circuits.py` at noise p, followed by a noiseless $T^\dagger$ and X readout of the decoded logical qubit.



Sahay et al. Fig. app_hxy(b): $|Y\rangle$ vs. $|H_{XY}\rangle$ cultivation, fault distance 3, no escape

Compute: not yet sampled

All detectors postselected; noiseless decoding and logical readout. The paper's figure is shown only as an image (no released numbers for this panel), so no overlay. As in the paper, the two states have different logical error rates once escape is excluded.

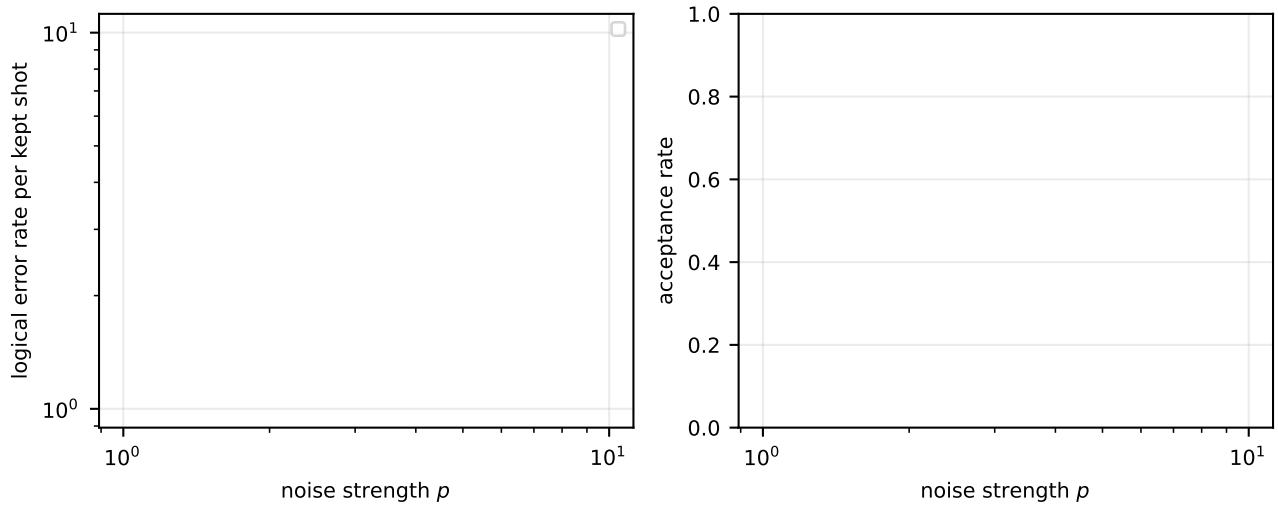
7 Chan, Strikis, Sun, Cai (2026), Diagnosing and restoring the degraded fault distance of magic state cultivation

arXiv:2609.17706 Study module `repro/studies/chan_flags.py`

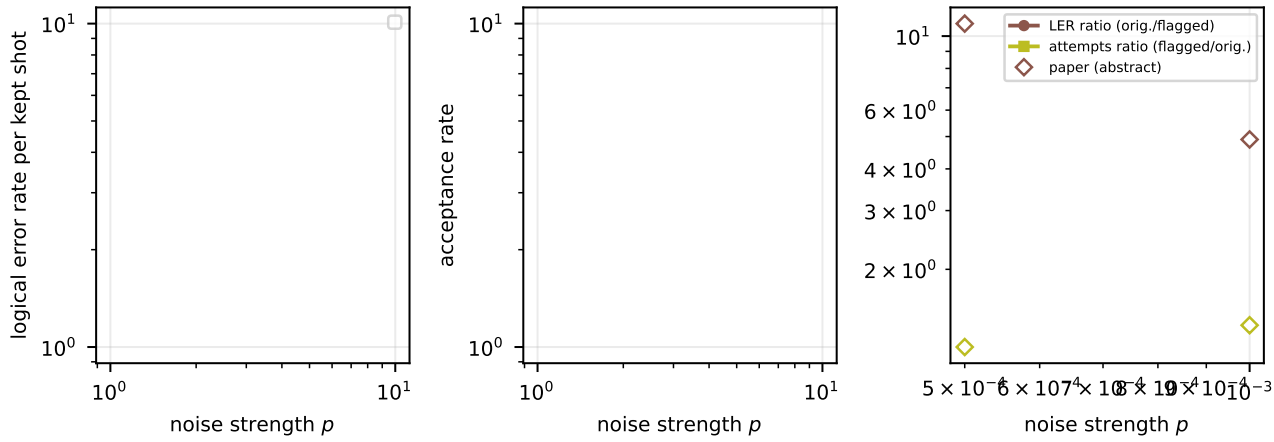
Pre-escape colour-code cultivation (injection + double checks, every detector postselected, noiseless final check), with and without the paper's Z flags in the final double check:

- Fig. `d3_pre_escape_ler`: $d = 3$ ($a = 6$ ancillas, $f = 2$ flags), S vs. T;
- Figs. `d5_pre_escape_ler` and `improvement_and_cost_ratios`: $d = 5$ ($a = 19$, $f = 13$), T state, original vs. flagged. The abstract quotes a 4.9x LER improvement at 1.36x attempts for $p = 10^{-3}$, and 10.9x at 1.17x for $p = 5 \cdot 10^{-4}$.

Circuits: the package's bundled S-state references (timchan0/cliffordea at 27385c6, `sim/stim_files`, the $d = 5$ ones with the Bell-growth feedforward and detector healing) and its own `make_variant_text`: `replace_noise_level` from 10^{-3} to p, and `swap_t_and_s_gates` for the T state. The paper sampled with SymFT.



Chan et al. Fig. d3_pre_escape_ler: $d = 3$ cultivation, S vs. T, with and without Z flags
Compute: not yet sampled
Blue: S proxy; orange: real T gates. Solid: the original double check ($a = 6$); dashed: with two Z flags ($f = 2$). The flags restore fault distance 3 for the T state.



Chan et al. Figs. d5_pre_escape_ler and improvement_and_cost_ratios: $d = 5$ T cultivation with Z flags

Compute: not yet sampled

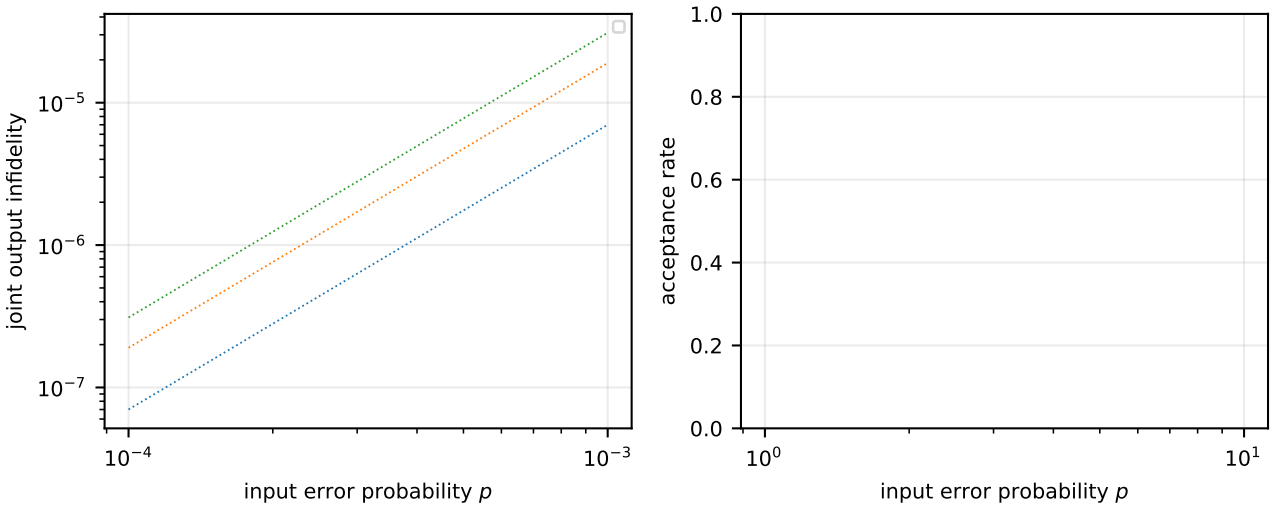
Pre-escape $d = 5$ T-state cultivation. Right: LER improvement and attempts ratio of the flagged circuit; open diamonds are the ratios quoted in the paper's abstract and Sec. VI.

8 Wang, Gu (2026), Magna: simulating magic state factories as cheaply as Clifford circuits

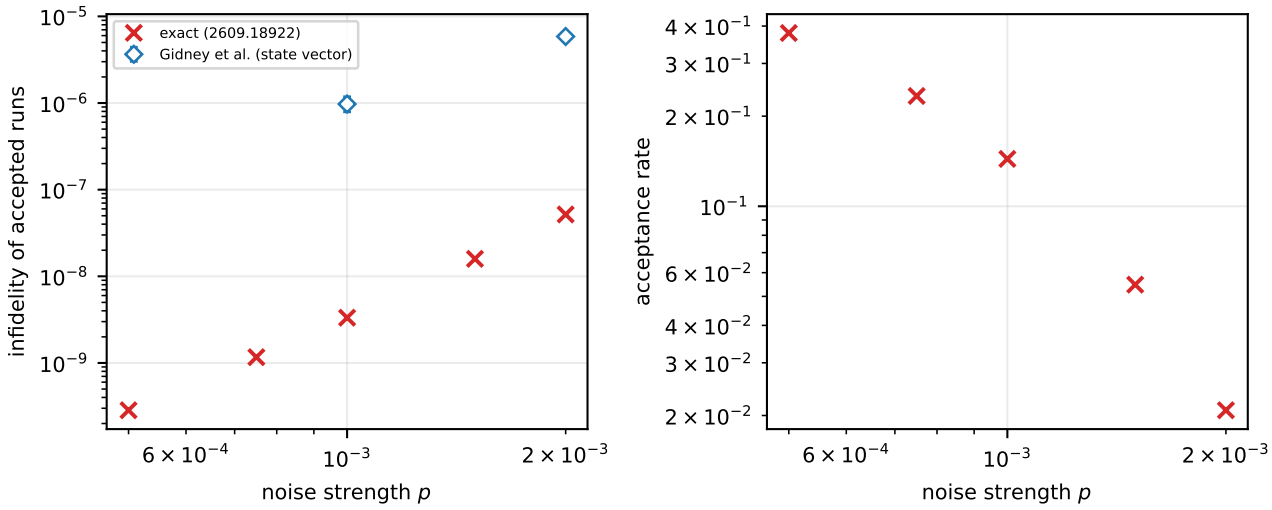
arXiv:2610.22315 Study module `repro/studies/magna.py`

- Fig. 8(a): Bravyi–Haah $(3k+8)$-to-k distillation on bare qubits, $k = 2, 6, 10$. Every input magic state has a Z error with probability p; everything else is noiseless. Plotted: the joint infidelity of the k outputs with $|T\rangle^{\otimes k}$, for p from 10^{-4} to 10^{-3}.
- Fig. 9(a,b): colour-code and RP^2 cultivation at $d = 3, 5$ (no escape), acceptance rate and infidelity of accepted runs under uniform depolarizing noise.

Circuits: Magna's own example protocols (itpyi/magna at 01ae9c2, `examples/*/physical.magna`), translated to the Tsim dialect by our `make_circuits.py` (deferred-measurement controlled corrections etc.). The examples are written at $p = 10^{-3}$ and Magna ships no generator, so other noise strengths are made by changing that literal. Only the detectors named in the examples' `POSTSELECT` lines are postselected. The error check is the examples' task definition: undo the ideal target circuit on the output qubits and require every output to read 0 (a joint check). Clifford versions: Magna's `-S` examples (product-S targets); for the colour code the S proxy is in the Gidney study.



Magna Fig. 8(a): Bravyi–Haah $(3k+8)$-to-k distillation
Compute: not yet sampled
Bare qubits, Z errors of probability p on the input magic states only. Dotted: the leading order $(1+3k)p^2$. Dashed/squares: the matched Clifford circuit ($|S\rangle$ inputs, product-S target).



Magna Fig. 9(a,b): colour-code and RP^2 cultivation at $d = 3, 5$
Compute: not yet sampled
Magna's cultivation examples (no escape). Markers as in the paper: Gidney et al.'s state-vector values for Color $d = 3$ and the exact values of arXiv:2609.18922 for Color $d = 5$. Dashed: Magna's S-target RP^2 examples.

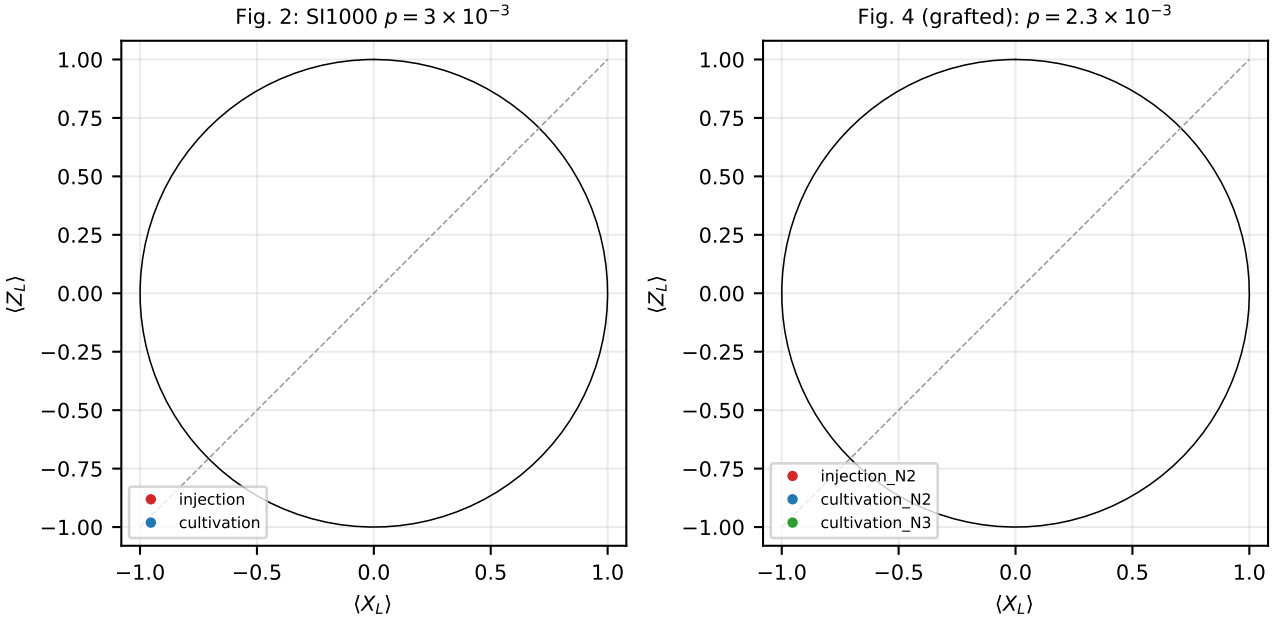
9 Rosenfeld et al. (2025), Magic state cultivation on a superconducting quantum processor

arXiv:2512.13908 Study module `repro/studies/google_experiment.py`

The paper's grey points are state-vector simulations of the experimental circuits under the SI1000 noise model; shots are kept only if no detector fires (post-selected logical tomography). Here:

- Fig. 2(b,c): logical tomography ($\langle X_L\rangle$, $\langle Z_L\rangle$) of injection only and of injection + cultivation, vs. the injected angle θ, SI1000 $p = 3.0 \times 10^{-3}$;
- Fig. 3(b): kickback tomography (KT) of the cultivated $|T\rangle$: logical error vs. θ, with and without the QEC cycles, SI1000 $p = 2.3 \times 10^{-3}$; SI Fig. `validating_KT`: KT error at the T angle vs. p (expected $\propto p^3$);
- Fig. 4(b): tomography after grafting to the surface code, injection only and cultivation with $N = 2, 3$ cycles, SI1000 $p = 2.3 \times 10^{-3}$.

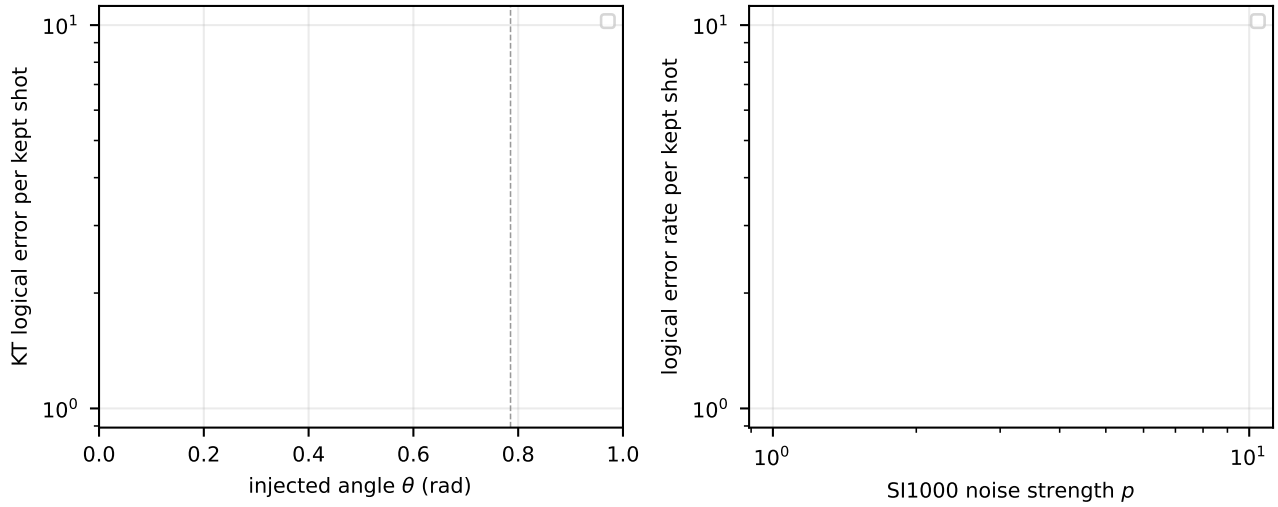
Circuits: the released Zenodo (17872003) circuits, repaired to real gates from their Cirq versions by our `make_circuits.py` (injection $e^{-i\theta Y/2}$ as an H_{YZ}-conjugated Z rotation, $T/T^\dagger$ checks). The injection rotation is set to $R_Z(\theta/\pi)$; SI1000 noise is added with Gidney's `gen.NoiseModel.si1000(p)` (the model the paper cites), with non-Clifford gates mapped to Clifford placeholders while the noise is inserted. The experiment had no Clifford proxy, so there is none here.



Rosenfeld et al. Figs. 2(b) and 4(b): post-selected logical tomography vs. injected angle

Compute: not yet sampled

Each marker is one injected angle θ (24 angles in $[0, 2\pi)$), with $\langle X_L\rangle$ and $\langle Z_L\rangle$ from the X- and Z-basis circuits (up to 3×10^9 shots each). Cultivation projects the state onto the magic axis (dashed). These replace the paper's grey state-vector points.



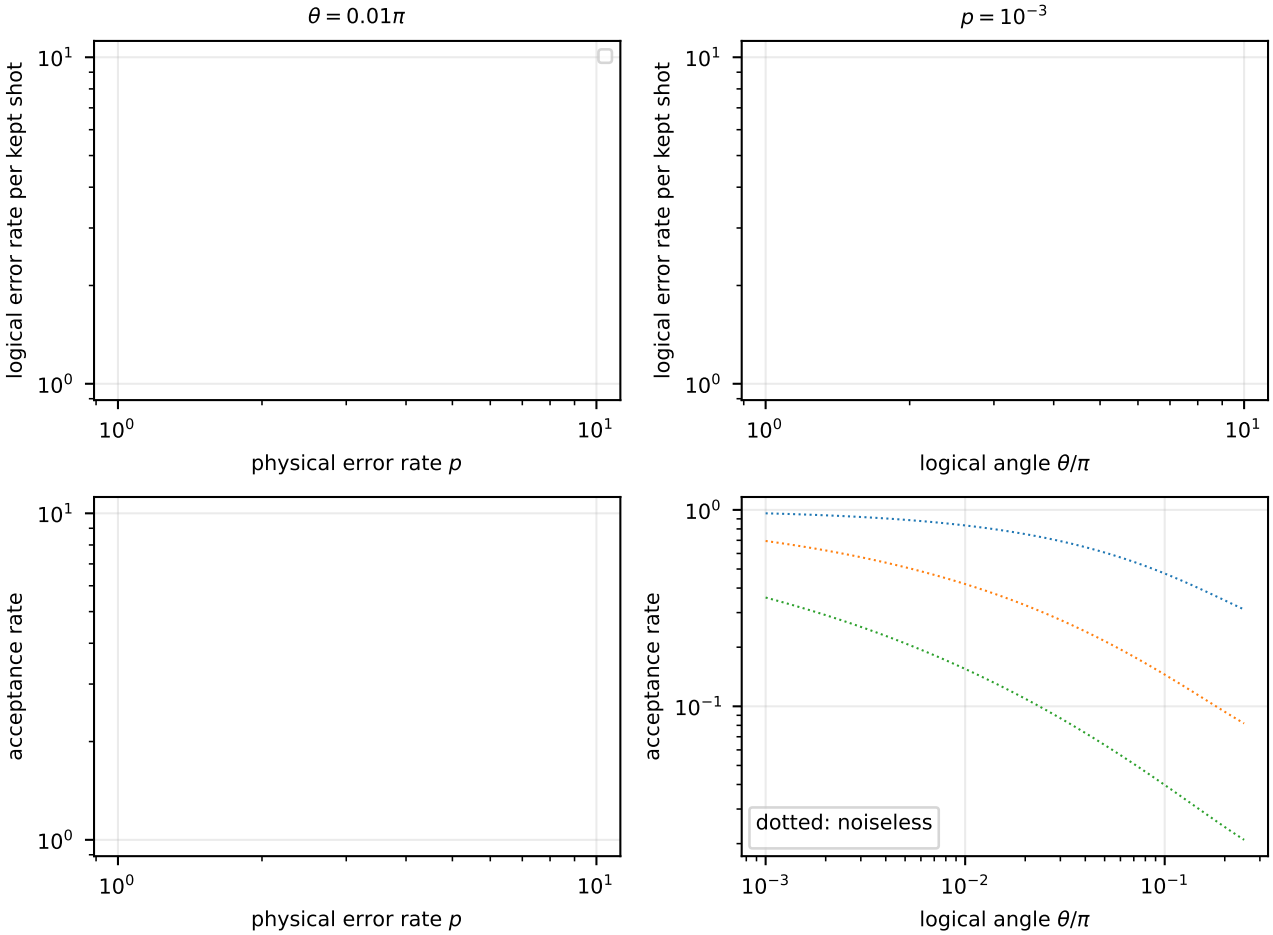
Rosenfeld et al. Fig. 3(b) and SI Fig. validating_KT: kickback tomography of the cultivated $|T\rangle$
Compute: not yet sampled
Left: KT logical error vs. injected angle (SI1000 $p = 2.3 \times 10^{-3}$), with and without the interleaved QEC cycles. Right: KT error at $\theta = \pi/4$ vs. p, which scales as p^3 (fault distance 3).

10 Ismail et al. (2025), Transversal architecture for megaquop-scale quantum simulation with neutral atoms (transversal STAR)

arXiv:2509.18294 Study module `repro/studies/star_injection.py`

The paper's Fig. 6 shows the logical error rate and success rate of its transversal multi-rotation (TMR) STAR injection with full postselection, vs. the physical error rate (at a logical angle of 10^{-2}) and vs. the angle; Fig. 13 validates against an analytic success rate at $d = 3$.

The authors released no generator; the circuits here are QuEra's public STAR challenge circuits (`QuEra-Challenge/star_cir` with the challenge's own `physical_angle` helper to set the logical angle. They differ from the paper's Fig. 6 protocol: $k = d$ physical rotations (the paper uses $k = 3$; equal at $d = 3$), d postselected rounds after the rotations, uniform depolarizing noise ($p = 10^{-3}$ in the released files; other p by changing that literal), and a noiseless logical un-rotation and X readout. Every detector is postselected (raw parity; the $d - 1$ detectors right after the rotations are random by design). The paper's Clifford approximation is not specified, so no Clifford version is sampled; the noiseless success probability $\cos^{2k}(\phi/2) + \sin^{2k}(\phi/2)$ of k physical rotations by ϕ is overlaid.



Ismail et al. Figs. 6, 13, 15 (protocol of the public STAR challenge circuits)
Compute: not yet sampled
Logical error of the injected small-angle rotation (noiseless un-rotation, logical X readout) and success rate, with every detector postselected. Note the protocol differences listed above ($k = d$, d postselected rounds after the rotations, uniform noise), so the curves are not expected to coincide with the paper's.

11 Chen et al. (2025), Efficient magic state cultivation on RP^2

arXiv:2503.18657 Study module `repro/studies/rp2_ungrown.py`

Table 1 lists, at $p = 10^{-3}$, the logical error rate and discard rate of each cultivation process alone (no escape stage), benchmarked by sampling the Clifford variant (S for T, cultivating $S|+\rangle$) with a final round of perfect stabilizer measurement and logical readout: MSC-3 8.3×10^{-7} / 48.6\%, MSC-5 3.6×10^{-10} / 90.5\%.

Circuits: the authors' `circuit_garage` files (Zihan-Chen-PhMA/Cultiv_T_RP2 at 77ad9b3, written at $p = 10^{-3}$ by their `d3_T_cult_sampling.py` / `d5_T_cult_sampling.py`). Clifford version: as released. T version: $S \to T$, $S^\dagger \to T^\dagger$ (reversing the authors' substitution) plus a noiseless logical rotation (`repro/magic_check.py`) so that the final Y_L readout checks the T state deterministically. MSC-5 with T gates has Tsim active width 10, which makes it the slowest circuit here.

Chen et al. Table 1 (ungrown columns): RP^2 cultivation without escape, $p = 10^{-3}$
Every detector postselected, perfect final stabilizer round and logical readout.
Compute: not yet sampled

	paper (S proxy)		Tsim GPU, S proxy			Tsim GPU, T		
protocol	LER	discard	LER [1000x interval] (errors)	discard	shots, time	LER (errors)	discard	shots, time
MSC-3	8.3e-07	48.6%	–	–	–	–	–	–
MSC-5	3.6e-10	90.5%	–	–	–	–	–	–

12 Jacoby et al. (2025), Magic state injection with erasure qubits

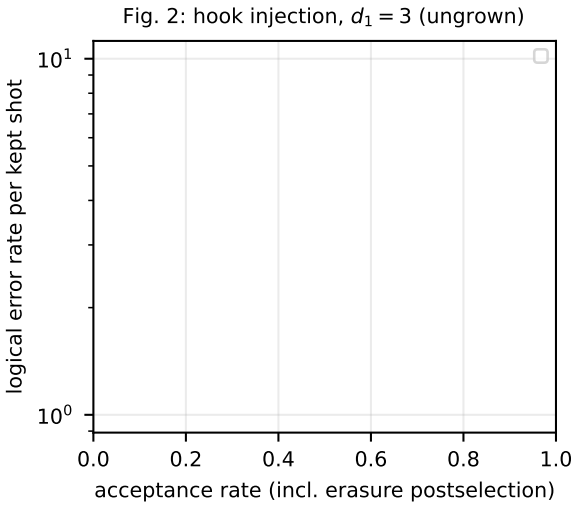
arXiv:2504.02935 Study module `repro/studies/erasure_injection.py`

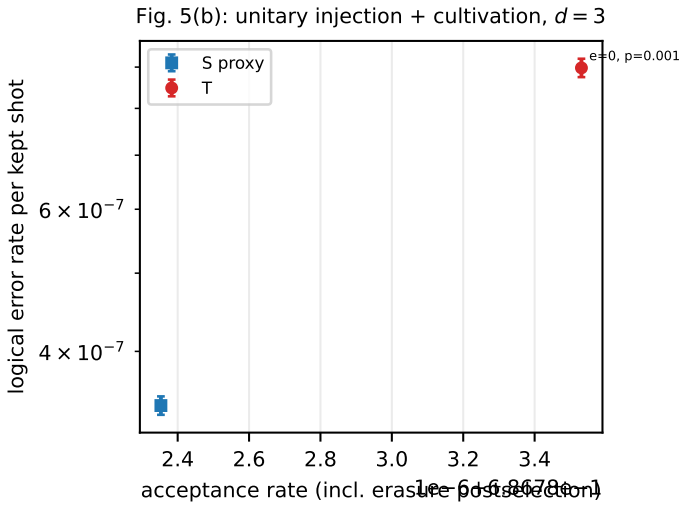
With erasure qubits, every heralded erasure in the (ungrown, fully postselected) patch discards the shot, so the logical error rate of kept shots depends only on the residual Pauli rate p, and the acceptance rate is the Pauli acceptance times the probability of no erasure. The paper's scenarios: $(e, p) = (0, 10^{-3})$, and $e \in \{10^{-3}, 4 \cdot 10^{-3}, 10^{-2}\}$ with $p = 10^{-4}$; erasure probability e per two-qubit gate and SPAM operation and $e/10$ per single-qubit gate (`end2end_script.py`: `e1=e/10, e_SPAM=e`).

Circuits:

- Fig. 5(b), unitary injection + $d = 3$ colour-code cultivation: Gidney's generator with uniform depolarizing noise (the paper's circuit is Gidney's `inject[unitary]+cultivate` file), S proxy and T; the $e = 0$ point is the Gidney study's $p = 10^{-3}$ point.
- Fig. 2, $d_1 = 3$ hook injection of $|S\rangle$: the released `circuits/QEC_hook_d=3.stim` ($p = 10^{-3}$; $p = 10^{-4}$ by scaling its noise literals, since the notebook that generated the paper's circuits is not released). T version: the injection $S \to T$ and a noiseless logical rotation (`repro/magic_check.py`) so the final Y_L readout checks the T state. Lao–Criger injection and the hybrid layouts are not reproduced.

The no-erasure probability is computed exactly from the Pauli circuit's noise locations, following `get_erased_circuit`.





Jacoby et al. Figs. 2 and 5(b): erasure qubits, ungrown injection and cultivation

Compute: not yet sampled

Each point is one erasure scenario (e, p); the logical error rate comes from the Pauli-only circuit at residual rate p (one sample serves every e), and the acceptance is the sampled Pauli acceptance times the exact no-erasure probability.

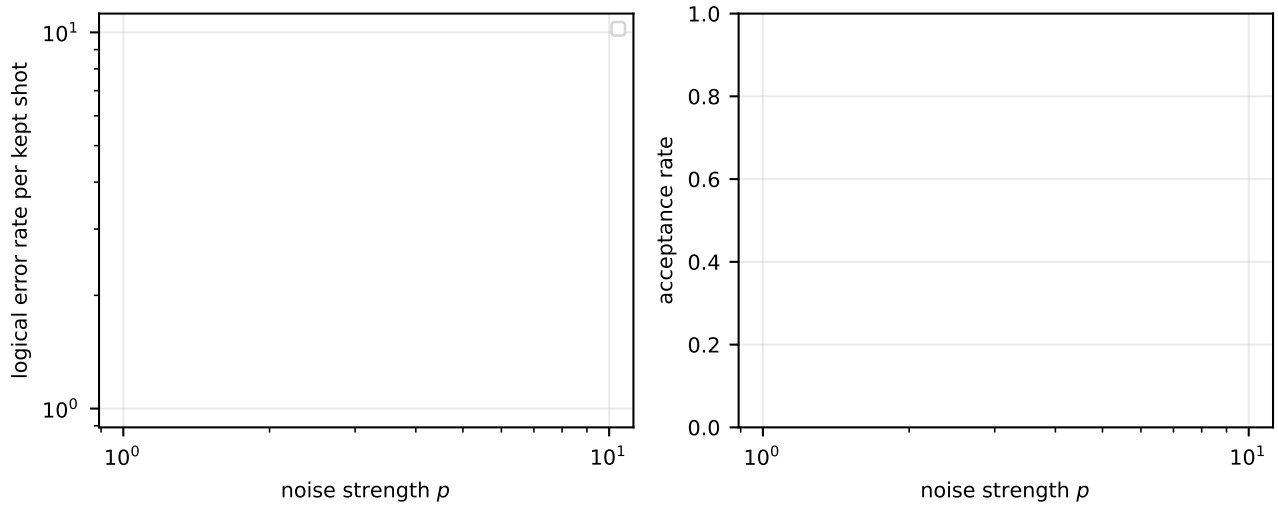
13 Vaknin et al. (2025), Efficient magic state cultivation on the surface code

arXiv:2502.01743 Study module `repro/studies/vaknin_full_vector.py`

Unrotated $d_1 = 3$ surface-code H-type cultivation (injection + double-checking with GHZ ancillas), without the expansion step: every stabilizer measurement postselected, then a perfect logical measurement in the magic-state basis. The authors spent about 10^6 CPU-hours on a full state-vector simulation, roughly 10^7 shots per point, and compared it with their Clifford approximation.

Circuits: the authors' generator (tomirendo/surface-code-magic-state-cultivation at ec28f6b, `Simulation(d=3)`: `generate_syndrome_circuit(detect_x=False)` + `generate_injection_circuit` + `generate_double_ghz`) in its own pinned environment (`.venvs/vaknin`, `repro/external/vaknin_generate.py`), with their `NoiseModel().uniform_d`

- Clifford approximation: `to_stim_circuit(apply_non_cliffords=False)` (non-Clifford gates replaced, their noise kept), final Z_L readout;
- non-Clifford: `apply_non_cliffords=True`, placeholders replaced by $T/T^\dagger$/CCZ/CH/CSWAP decompositions (our `make_circuits.py`); the final noiseless readout is set to Z_L, preceded by the noiseless logical rotation $\exp(\mp i\pi Y_L/8)$ (`repro/magic_check.py`) that turns the final Z_L readout into the magic-basis (H_L) check.



Vaknin et al. Fig. 26: unrotated H cultivation, non-Clifford vs. Clifford approximation
Compute: not yet sampled
All stabilizers postselected, no expansion, perfect magic-basis logical readout. The paper's full state-vector run took about 10^6 CPU-hours for $\sim 10^7$ shots per point; its numbers are shown only in the figure, so there is no overlay.